

Task-1

Roll No: 241212 | IIT Kanpur ACA Summer Project - GanForge

Summary

In this task we have to capture an image using Webcam and write python functions for:

- Converting the image to the HSV color space
- Applying histogram equalization for contrast enhancement
- Apply binary inversion thresholding
- Reducing the image to 4 gray intensity levels
- Applying Laplacian and Scharr filter
- Applying median filter
- Applying unsharp masking for sharpening
- Converting the image from RGB to LAB color space

Capturing the image using OpenCV

`cv2.VideoCapture(0)` accesses the default webcam. `cap.read()` is used to capture one frame. `cv2.imwrite()` saves it for further processing.

Converting to HSV Color Space

`cv2.cvtColor(img, cv2.COLOR_BGR2HSV)` converts the image from BGR to HSV. It is BGR because OpenCV loads images in BGR format by default, not RGB. `cv2.split(hsv)` splits the 3-channel HSV image into three separate grayscale images.

Channel type	What does it do
Hue	shows dominant colors
Saturation	shows how pure the color is
Value	shows brightness levels

Histogram Equalization

Histogram of a grayscale image shows how many pixels have each intensity value from 0 to 255. It redistributes pixel intensities to cover a wider range, making dark areas darker and bright areas brighter — revealing hidden details helpful in edge detection and feature extraction.

```
cv2.COLOR_BGR2GRAY – converts image to Grayscale
```

```
cv2.equalizeHist(gray) – applies histogram equalization to enhance contrast
```

Binary Inversion

General form: `cv2.threshold(src, thresh, maxval, type)`

Parameter	Description
src	input image (must be grayscale)
thresh	Threshold value to compare pixel intensity
maxval	Value to assign if the condition is satisfied
type	binary / inverse / truncation..

Syntax used: `cv2.threshold(gray, 127, 255, cv2.THRESH_BINARY_INV)`. Pixels with intensity > 127 become 0; pixels <= 127 become 255. Useful for text segmentation — dark text on light background gets inverted.

Posterization

```
posterized = (gray // 64) * 64
```

Reduces the number of gray levels to 4.

Edge Detection using Laplacian and Scharr Filters

Laplacian detects rapid intensity changes (edges). It is a second-order derivative filter that calculates the second-order derivative of the image and detects regions of rapid intensity change. It is isotropic — responds equally to edges in all directions.

CV_64F is used because derivatives can produce negative values, and uint8 cannot store negatives — so a floating point format is used to preserve the result.

`cv2.Scharr(gray, cv2.CV_64F, 1, 0)` detects vertical edges (X-direction). Use (0, 1) instead of (1, 0) to detect horizontal edges.

Median Filtering

```
noisy = gray.copy()
```

Creates a copy of the grayscale image so the original does not change. Noise is added manually:

```
noisy[::10, ::10] = 0    # every 10th row & col set to black
noisy[::15, ::15] = 255  # every 15th row & col set to white
```

cv2.medianBlur(noisy, 3) removes salt-and-pepper noise effectively.

Unsharp Masking

Enhances edges and fine details by subtracting a blurred version from the original.

```
blurred = cv2.GaussianBlur(gray, (5,5), 0)
```

gray is the input grayscale image; (5,5) is the kernel size (must be odd so the kernel can be centered on a single pixel). The central pixel becomes the focus for calculating the new value.

```
sharpened = cv2.addWeighted(gray, 1.5, blurred, -0.5, 0)
```

Equivalent to: $\text{sharpened} = 1.5 \times \text{original} - 0.5 \times \text{blurred} + 0$. This adds back the detail removed by the blur, sharpening the edges.

Unsharp mask formula: $\text{sharpened} = \text{original} + \text{amount} \times (\text{original} - \text{blurred})$

LAB Color Space

Powerful for color-based processing such as color enhancement and object segmentation.

Channel	Meaning	Value range
L	Lightness	0 to 255
A	Green <-> Red	0 to 255 (Centered at 128)
B	Blue <-> Yellow	0 to 255 (Centered at 128)

```
lab = cv2.cvtColor(img, cv2.COLOR_BGR2LAB) # BGR to LAB conversion
```

```
L, A, B = cv2.split(lab) # separates the 3 LAB channels
```

Output: Image Processing Results

The following figure shows the output of all 8 image processing operations applied to the sample image (Roll No: 241212):

